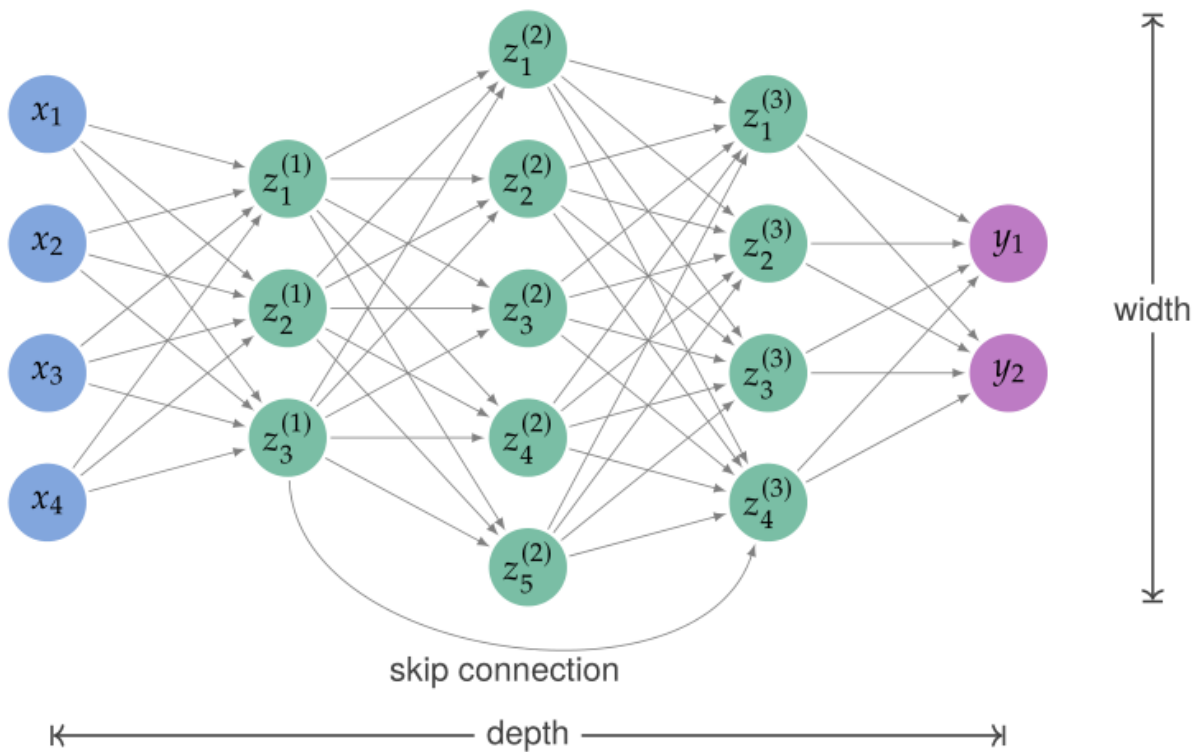


MEMORIA - TAREA 8: REDES NEURONALES CONVOLUCIONALES



Nombre y apellidos: Ruben Perez Gonzalez

Asignatura: Sistemas de Aprendizaje Automático

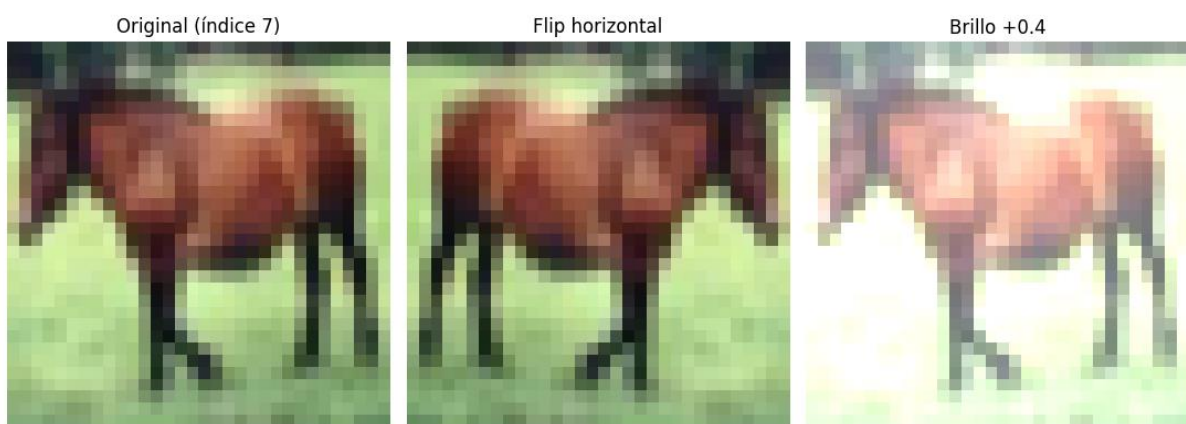
Fecha: 13/05/2026

1. OBJETIVO

En esta práctica se implementa una red neuronal convolucional (CNN) desde cero y se compara con un enfoque de Transfer Learning utilizando MobileNetV2. También se analiza el efecto del Data Augmentation (flip, brillo y rotación) para entender la invarianza y robustez del modelo.

2. PARTE 1 - LABORATORIO DE DATOS E INVARIANZA

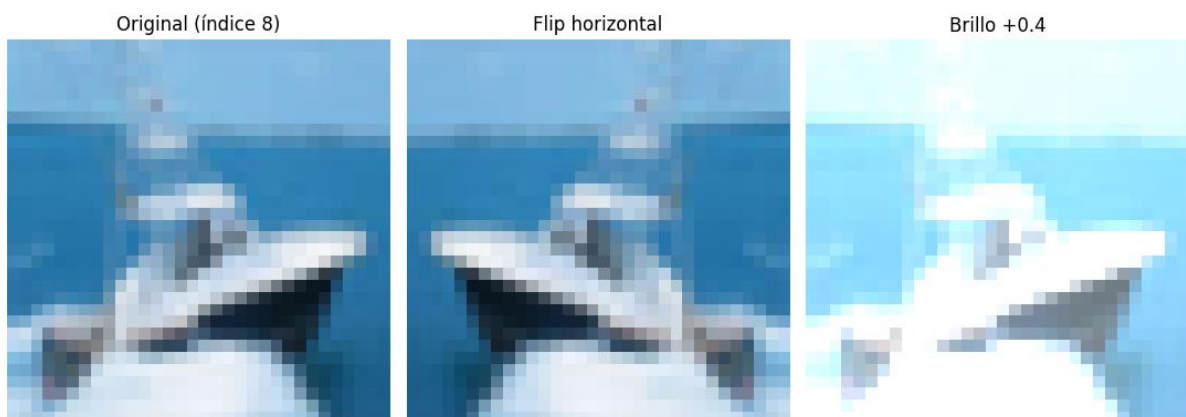
2.1 Experimento base (índice 7)

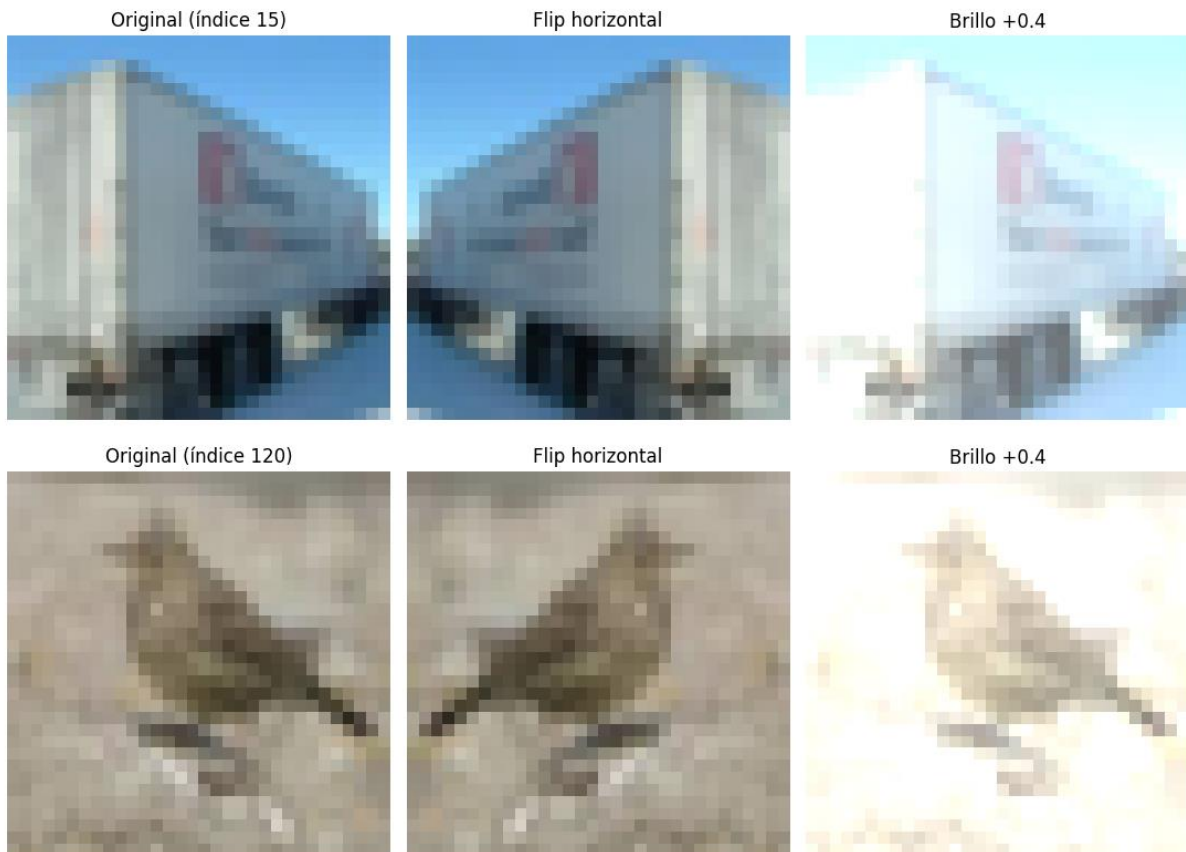


Respuesta 1:

- Sí, un humano sigue reconociendo el objeto (un caballo) tras el volteo horizontal en la mayoría de casos.
- Para la red neuronal, aunque el objeto sea el mismo conceptualmente, los píxeles no están en las mismas posiciones. Por tanto, la entrada numérica cambia.

2.2 Prueba con índices 8, 15 y 120





Respuesta 2.a: Sí, todos los objetos (el barco, el camión y el pájaro) siguen siendo perfectamente reconocibles tras el flip horizontal.

Respuesta 2.b: El aumento de datos parece más natural en elementos de la naturaleza (como el pájaro) o en elementos con alta simetría (el barco de frente). En el camión la forma es natural, pero si tuviera texto o logotipos escritos en el lateral (se aprecian ligeramente), se verían al revés y perdería naturalidad.

Respuesta 2.c: Los objetos más sensibles al volteo son aquellos que tienen lateralidad estricta o información direccional clave: textos, señales de tráfico, vehículos asimétricos o imágenes con posiciones de conducción definidas (volante a un lado).

2.3 Prueba con brillo -0.3, 0.1 y 0.8



Respuesta 3.a: Cuando el brillo es demasiado alto (+0.8), gran parte de la imagen se satura y los píxeles se acercan al blanco puro, perdiendo los contornos.

Respuesta 3.b: La imagen empieza a perder información útil cuando se eliminan los detalles de textura o contorno, ya sea por saturación al sobreexponer (se ve todo blanco) o por oscurecimiento excesivo (se vuelve negro).

Respuesta 3.c: Sí, una red neuronal podría confundirse con imágenes demasiado brillantes u oscuras porque cambian drásticamente los valores de los píxeles y desaparecen los patrones, bordes y texturas que la red utiliza para realizar la clasificación.

2.4 Prueba de rotación (90 grados)



Respuesta 4.a: Sí, un humano sigue reconociendo claramente que es un caballo, aunque detecte que la orientación no es la natural.

Respuesta 4.b: No siempre. Una rotación de 90 grados es útil en dominios sin orientación fija (como fotos satelitales, aéreas o microscópicas), pero en otros dominios introduce ejemplos poco realistas y sin sentido físico (como animales desafiando la gravedad).

Respuesta 4.c: Sería mala idea para imágenes donde la orientación es fundamental, por ejemplo: personas caminando, coches circulando por la carretera, paisajes con el cielo arriba, texto o señales de tráfico.

2.5 Caso de matrículas Respuesta 5: No sería buena idea usar volteo horizontal para matrículas. Al voltear, se invertiría el orden y la forma de las letras y números (se leería en formato espejo), generando datos completamente inválidos e irreales para un sistema de reconocimiento de caracteres.

3. PARTE 2 - CNN DESDE CERO

Arquitectura usada:

- Input (32,32,3)
- RandomFlip(horizontal)
- RandomRotation(0.1)
- Conv2D + MaxPooling2D
- Conv2D + MaxPooling2D
- Flatten + Dense + Dense(10, softmax)

3.1 Entrenamiento base

```
Epoch 1/2
782/782 ————— 12s 14ms/step - accuracy: 0.4136 - loss: 1.6247
Epoch 2/2
782/782 ————— 15s 19ms/step - accuracy: 0.5174 - loss: 1.3568
CNN base -> loss: 1.2250, accuracy: 0.5565, tiempo: 27.19s
```

Resultados base:

- Loss (test): 1.2250

- Accuracy (test): 0.5565
- Tiempo aproximado: 27.19 segundos

Pregunta 1 (MaxPooling2D): MaxPooling2D sirve para reducir las dimensiones espaciales de los mapas de activación (resolución), lo que disminuye drásticamente el coste computacional en capas posteriores, reduce el riesgo de sobreajuste y ayuda a extraer los rasgos más dominantes haciendo la red invariante a pequeñas traslaciones.

Pregunta 2 (RandomFlip y RandomRotation): Aportan la ventaja de aumentar artificialmente el dataset, mostrando a la red versiones ligeramente distintas de las mismas imágenes en cada época. Esto actúa como regularización, mejorando la generalización de la red y evitando el sobreajuste (overfitting).

3.2 Cambio de filtros (16, 64, 128)

```
Entrenando modelo con 16 filtros iniciales...
Epoch 1/2
782/782 ————— 8s 9ms/step - accuracy: 0.4029 - loss: 1.6519
Epoch 2/2
782/782 ————— 11s 14ms/step - accuracy: 0.4983 - loss: 1.4011

Entrenando modelo con 64 filtros iniciales...
Epoch 1/2
782/782 ————— 40s 48ms/step - accuracy: 0.4206 - loss: 1.6025
Epoch 2/2
782/782 ————— 54s 69ms/step - accuracy: 0.5364 - loss: 1.3074

Entrenando modelo con 128 filtros iniciales...
Epoch 1/2
782/782 ————— 120s 149ms/step - accuracy: 0.4291 - loss: 1.5777
Epoch 2/2
782/782 ————— 108s 139ms/step - accuracy: 0.5486 - loss: 1.2765

Resultados (filtros, loss, accuracy, tiempo_s):
(16, 1.297805905342102, 0.55059992275238, 19.07852339744568)
(64, 1.1616125106811523, 0.5873000025749207, 94.82817363739014)
(128, 1.1330136060714722, 0.5992000102996826, 228.94475317001343)
```

Tabla de resultados:

- 16 filtros -> Loss: 1.297805 | Accuracy: 0.550599 | Tiempo: 19.07852 s
- 64 filtros -> Loss: 1.16161 | Accuracy: 0.58730 | Tiempo: 94.8281 s

- 128 filtros -> Loss: 1.13301 | Accuracy: 0.5992 | Tiempo: 228.944 s

Respuesta 3.a: Al aumentar el número de filtros (de 16 a 128), el tiempo de entrenamiento aumenta considerablemente, ya que la red tiene que realizar muchas más multiplicaciones matriciales en cada paso.

Respuesta 3.b: No, más filtros no siempre significan mejor precisión. Una red con demasiados filtros tiene mucha capacidad de memorización y puede sufrir rápidamente de sobreajuste (overfitting), rindiendo peor en el conjunto de test. Además, puede necesitar más épocas o datos para aprovechar realmente esa capacidad.

Respuesta 3.c: Existe un compromiso (trade-off) claro: para alcanzar una mayor precisión suele ser necesario un modelo más grande, pero esto dispara el coste computacional (más memoria y mayor tiempo de entrenamiento y predicción).

3.3 Experimento sin MaxPooling2D

```
Epoch 1/2
782/782 ————— 55s 69ms/step - accuracy: 0.4348 - loss: 1.5780
Epoch 2/2
782/782 ————— 54s 69ms/step - accuracy: 0.5404 - loss: 1.2989
Sin pooling -> loss: 1.1735, accuracy: 0.5837, tiempo: 109.10s
Con pooling -> loss: 1.2250, accuracy: 0.5565, tiempo: 27.19s
```

Respuesta 4.a: Sin las capas de pooling, el tamaño espacial de las activaciones (la resolución de los mapas de características) no se reduce tras cada convolución. Por lo tanto, se mantienen grandes hasta llegar a la capa Flatten.

Respuesta 4.b: Sí, el entrenamiento tarda mucho más, porque al llegar a la capa Flatten con un mapa de activación tan grande, la cantidad de parámetros en la capa Dense que le sigue se multiplica enormemente, aumentando las operaciones matemáticas.

Respuesta 4.c: Las CNN modernas siguen usando reducción espacial (pooling o strides) para mantener bajo control el coste computacional, ampliar el campo receptivo de las neuronas más profundas (permitiéndoles "ver" zonas más grandes de la imagen original) y capturar jerarquías de características de lo local a lo global.

4. PARTE 3 - TRANSFER LEARNING (MOBILENETV2)

Configuración:

- Modelo base: MobileNetV2 pre-entrenado en ImageNet
- include_top=False

- `base_model.trainable=False`
- Cabeza nueva: `GlobalAveragePooling2D + Dense(10, softmax)`
- Epochs: 5

Model: "sequential_11"

Layer (type)	Output Shape	Param #
mobilenetv2_1.00_32 (Functional)	(None, 1, 1, 1280)	2,257,984
global_average_pooling2d_1 (GlobalAveragePooling2D)	(None, 1280)	0
dense_21 (Dense)	(None, 10)	12,810

Total params: 2,270,794 (8.66 MB)

Trainable params: 12,810 (50.04 KB)

Non-trainable params: 2,257,984 (8.61 MB)

Epoch 1/5

782/782 ————— 43s 49ms/step - accuracy: 0.2514 - loss: 2.1270

Epoch 2/5

782/782 ————— 31s 39ms/step - accuracy: 0.2795 - loss: 2.0134

Epoch 3/5

782/782 ————— 48s 49ms/step - accuracy: 0.2840 - loss: 1.9819

Epoch 4/5

782/782 ————— 36s 46ms/step - accuracy: 0.2864 - loss: 1.9668

Epoch 5/5

782/782 ————— 31s 39ms/step - accuracy: 0.2889 - loss: 1.9578

Transfer Learning -> loss: 1.9689, accuracy: 0.2834, tiempo: 188.63s

Accuracy CNN Propia: 0.5565 | Accuracy Transfer Learning: 0.2834

Resultados Transfer Learning:

- Accuracy CNN propia: 0.5648
 - Accuracy Transfer Learning: 0.2838
1. ¿Cuál de los dos modelos ha obtenido mejor precisión? La CNN implementada desde cero ha obtenido una precisión mucho mejor (0.5648) frente al modelo de Transfer Learning (0.2838).
 2. ¿Por qué el Transfer Learning obtiene peor accuracy que la CNN propia? ¿Dónde estamos fallando en el código? Estamos fallando porque las imágenes originales probablemente son de 32x32 (tipo CIFAR-10), un tamaño demasiado pequeño para los filtros de MobileNetV2, que fue entrenado para recibir imágenes de 224x224. Además, solo estamos entrenando 5 épocas una única capa densa final mientras el resto del modelo está congelado. *Mejora:* Para obtener mejores resultados, deberíamos hacer un redimensionamiento (resize) de la imagen a un tamaño mayor antes de pasarla al modelo base, aumentar el número de épocas o hacer un ligero "fine-tuning" descongelando las últimas capas convolucionales.
 3. ¿Qué significa que hayamos puesto `base_model.trainable = False`? Significa que estamos "congelando" todos los pesos (parámetros) pre-entrenados del modelo base MobileNetV2 (sus 2.2 millones de parámetros no entrenables). Durante el entrenamiento, estos pesos no se actualizarán; la red solo aprenderá y modificará los pesos de la nueva capa Dense clasificadora que hemos añadido al final (los 12,810 parámetros entrenables).
 4. Cambia `base_model.trainable = False` por `base_model.trainable = True`. Contesta: a)
¿El entrenamiento tarda más? Sí, tarda muchísimo más. Al estar en "True", el algoritmo de backpropagation debe calcular gradientes y actualizar todos y cada uno de los más de 2 millones de parámetros del modelo base en cada iteración.
- b) ¿La precisión mejora siempre? No. Descongelar todo el modelo de golpe, especialmente si usamos una tasa de aprendizaje (learning rate) alta típica de un entrenamiento desde cero, puede destruir los valiosos pesos pre-entrenados ("olvido catastrófico") y acabar con peores resultados.
- c) ¿Por qué entrenar millones de parámetros puede ser peligroso con pocos datos? Porque un modelo gigantesco (con millones de parámetros) tiene una capacidad enorme de memorizar los datos de entrenamiento a la perfección. Si el conjunto de datos es pequeño, el modelo sobreajustará (overfitting) rápidamente, aprendiendo ruidos específicos de esos datos en lugar de características generales, y fracasará al enfrentarse a imágenes nuevas.

5. CONCLUSIONES

- El Data Augmentation ayuda a que el modelo sea más robusto frente a cambios visuales moderados.
- Una CNN propia permite entender la arquitectura y los efectos de parámetros como filtros y pooling.
- El Transfer Learning suele ofrecer un mejor punto de partida al reutilizar características aprendidas en grandes conjuntos de datos, siempre y cuando se configure adecuadamente el tamaño de entrada y el proceso de fine-tuning.
- La elección final depende del equilibrio entre precisión, tiempo y recursos disponibles.